

1. Authentication & User Management (Unified RBAC)

All users interact with a single entry point, but their experience changes based on their **role** (Student, Setter, Admin).

- **Registration & Login:**
 - Secure signup using email/password via **BetterAuth**.
 - JWT-based session management with "Remember Me" functionality.
 - **Password Reset:** Email-based token recovery system.
- **Profile Management:**
 - **Update Profile:** Edit display name, bio, and profile picture (stored in Cloudinary/DB).
 - **Role Request:** Students can submit a "Problem Setter Request" form which includes a brief motivation.

2. Student Features (The Solver)

A. Problem Discovery

- **Search:** Real-time partial-match search by Problem Title or Topic Tag.
- **Filtering:**
 - **Difficulty:** Toggle buttons for Easy, Medium, and Hard.
 - **Status:** Filter by "Solved," "Attempted," or "Unsolved."
- **Sorting:** Sort the table by Acceptance Rate (ASC/DESC), Difficulty (Low to High), or Date Added.
- **Pagination:** Standard view of **20 problems per page** to ensure fast loading times.

B. The Workspace & Execution

- **Code Editor:** Integration of the **Monaco Editor** with C++ syntax highlighting, auto-completion, and basic indentation.
- **Submission Engine:**
 - **Function:** Code is sent to the backend, placed in an **Inngest queue**, and executed in a Docker sandbox.
 - **Verdicts:** Return specific errors: **Accepted (AC)**, **Wrong Answer (WA)**, **Time Limit Exceeded (TLE)**, **Runtime Error (RE)**, or **Compilation Error (CE)**.
 - **Resource Stats:** Display Execution Time (ms) and Memory Usage (MB) for all successful runs.
- **Submission History:** A personal log showing all previous attempts for that specific problem with timestamps and verdicts.

C. Statistics & Gamification

- **Student Dashboard:**
 - **Solved Count:** Breakdown by difficulty (e.g., "15 Easy, 5 Medium, 0 Hard").
 - **Success Rate:** Calculation: $\frac{\text{Accepted Submissions}}{\text{Total Submissions}} \times 100\%$.
 - **Activity Heatmap:** A GitHub-style contribution graph showing solving activity over the last 12 months.

- **Global Leaderboard:** Rankings based on a point system (e.g., Easy=10pts, Med=30pts, Hard=50pts) with pagination for top 100 users.
 - **Bookmarks:** A "Save for Later" list for favorite or challenging problems.
-

3. Problem Setter Features (The Creator)

A. Problem Studio

- **Creation Form:**
 - **Content:** Rich-text editor for the problem description, including LaTeX support for math formulas.
 - **Specifications:** Explicit fields for Input Format, Output Format, and Constraints (e.g., "Time Limit: 1.0s, Memory: 256MB").
 - **Test Case Management:**
 - **Sample Cases:** Visible to students in the problem description.
 - **Hidden Cases:** Strictly for evaluation; must be uploaded as `.txt` files or entered via multiple input/output fields.
- **Video Solution (Optional):** Upload a video file (MP4) to Cloudinary. The video only becomes accessible to students after they solve the problem or via a "View Solution" button (Admin-configurable).

B. Management & Analytics

- **Status Tracker:** View the lifecycle of a problem: `Draft` $\rightarrow$ `Pending Review` $\rightarrow$ `Approved` or `Rejected`.
 - **Feedback Loop:** View a "Rejection Reason" provided by the Admin to fix and resubmit.
 - **Performance Metrics:**
 - Total submissions received for their problems.
 - Acceptance rate per problem (helps setters identify if a problem is too hard/easy).
 - View anonymized student code for their problems to understand common mistakes.
-

4. Admin Features (The Moderator)

A. Content Moderation

- **Review Queue:** A dashboard showing all "Pending" problems from Setters.
- **Approval Workflow:**
 - **Function:** Admin can edit the problem for typos, test the code against their own solution, and click "Approve" (makes it public) or "Reject" (notifies the Setter).
- **Management:** Ability to "Feature" a problem (pin to top) or delete inappropriate content.

B. User & Platform Control

- **Role Management:** Approve or deny Student-to-Setter requests.

- **Account Controls:** Reset passwords, ban users for cheating, or delete accounts.
 - **Platform Health Dashboard:**
 - Real-time counters for Total Users, Total Problems, and Active Users this week.
 - Global Activity Feed: "User X just solved Problem Y," "Setter Z submitted a new problem".
-

5. Contest System (The Competition)

- **Contest Creation:** Admin or Setters can create a contest by selecting a set of problems and defining a Start/End time.
- **Real-time Leaderboard:** A dynamic ranking during the contest that updates as students submit code.
- **Post-Contest Analysis:** Final standings and the ability for students to solve the contest problems in "Practice Mode" after it ends.

Features to put in the slide:

Slide 1: Student Features (The Solver)

- **Discovery:** Search by title/topic with Easy/Medium/Hard filters and status tracking (Solved/Unsolved).
- **Workspace:** Monaco Code Editor with syntax highlighting and auto-completion.
- **Execution:** Real-time verdicts (AC, WA, TLE, RE, CE) with execution time and memory usage stats.
- **Analytics:** Personalized dashboard with progress charts, success rate ($\frac{\text{Accepted}}{\text{Total}} \times 100\%$), and difficulty breakdown.
- **Social:** Competitive Global Leaderboard and live Coding Contests.

Slide 2: Problem Setter Features (The Creator)

- **Problem Studio:** Full lifecycle management (Draft $\rightarrow$ Pending $\rightarrow$ Approved).
- **Rich Content:** LaTeX-supported descriptions, sample cases, and hidden test case management.
- **Video Integration:** Optional video solution uploads via Cloudinary for advanced walkthroughs.
- **Insights:** Review student code for your own problems and track problem-specific acceptance rates.
- **Feedback Loop:** Receive and address Admin feedback for rejected problem drafts.

Slide 3: Admin Features (The Governor)

- **Content Moderation:** Centralized queue to review, edit, approve, or reject pending problems.
- **User Authority:** Manage role elevations (Student to Setter), account bans, and password resets.
- **System Dashboard:** High-level platform stats including total users, active sessions, and submission volume.
- **Contest Management:** Create and schedule timed contests with specific problem sets.

Technologies:

Technology	Where it's used	Why we are using it
React.js	Frontend UI	Provides a fast, component-based single-page application (SPA) experience for students and admins.
Node.js / Express	Backend API	Handles routing, business logic, and communication between the frontend and database.
MongoDB	Database	Flexible schema allows us to store complex problem data, diverse test cases, and student statistics easily.
Monaco Editor	Code Workspace	The same engine behind VS Code; gives students professional features like syntax highlighting and auto-indentation.
BetterAuth	Authentication	Simplifies secure session management and handles our Role-Based Access Control (RBAC) requirements out of the box.
Inngest	Background Queue	Acts as the "Reliability Layer." It handles long-running code execution tasks without blocking or crashing our main API.
Docker	Execution Sandbox	Creates a secure, isolated "room" to run student code, protecting our server from malicious scripts or infinite loops.

Cloudinary	Media Storage	Handles the heavy lifting of uploading, optimizing, and streaming video solutions for problem setters.
Tailwind CSS	Styling	Allows for rapid, responsive UI development so the platform looks modern on both desktop and mobile.

Workflow:

 **Data Modeling** --->  **Unified Auth (BetterAuth)** --->  **User Profiles student Experience (Frontend-First)** --->  **Problem Discovery UI** --->  **Monaco Editor Integration** --->  **Student Backend** --->  **Setter Frontend (Forms)** --->  **Setter Backend (CRUD)** --->  **Admin Moderation UI** --->  **Inngest Event Trigger** --->  **Docker Sandbox Logic** --->  **The Grader Utility** --->  **Contest System** --->  **Video Solutions** --->  **Final Stats & Leaderboards**

Phase 1: Foundation & Data Modeling

Don't write a single line of UI until the "brain" of the app is defined.

- **Database Schema Design:** Define the **MongoDB** models for **Users** (with roles), **Problems** (with test cases), and **Submissions**.
- **Environment Setup:** Initialize the **Express** backend, **BetterAuth** for security, and the **Inngest** client.
- **Monaco Integration:** Set up the frontend code editor early so you can test how code strings are sent to the backend.

Phase 2: The "Admin-to-Setter" Pipeline

You can't solve problems if there aren't any in the database.

- **Setter Workflow:** Build the form for creating problems (Title, Description, Constraints, and Sample/Hidden test cases).
- **Admin Moderation:** Create the dashboard where an Admin can view "Pending" problems and toggle their status to "Approved".
- **Access Control:** Implement middleware to ensure only **Admins** can approve and only **Setters** can create content.

Phase 3: The Core Judge Engine (The MVP)

This is the most critical phase. If this fails, the project fails.

- **The Worker Logic:** Create the **Inngest** function that listens for a **code.submitted** event.
- **Docker Integration:** Write the script that:
 1. Generates a temporary **.cpp** file from the user's code.

2. Spawns a **Docker container** to compile and run that file against the test cases.
 3. Captures `stdout` and monitors for Time/Memory limits.
- **The Grader:** Build the utility that compares the output against the "Hidden Test Cases" and returns the verdict (AC, WA, TLE, etc.).

Phase 4: Student Experience & Results

- **Submission Tracking:** Once the Judge finishes, update the `Submissions` collection in MongoDB.
- **Real-time Updates:** Implement a way for the frontend to know the result is ready (either through Inngest callbacks or polling).
- **Stats Engine:** Create the logic to calculate a student's **Success Rate** and **Difficulty Breakdown** based on their submission history.

Phase 5: Advanced Modules (The "Extra Mile")

- **Video Solutions:** Integrate the **Cloudinary** upload widget for Setters and the video player for Students.
- **Contest System:** Build the "Timed" logic—problems only become visible at a certain time, and a live **Leaderboard** tracks scores until the end time.
- **Leaderboard Analytics:** Aggregate global data to show rankings and "Active users this week" on the Admin dashboard.

Phase 6: Testing & Security Hardening

- **Security Audit:** Try to "break" your own judge. Submit infinite loops or malicious `system()` calls to ensure the Docker sandbox holds up.
- **Edge Case Testing:** Test the judge with empty inputs, very large strings, and complex C++ libraries.
- **UI/UX Polish:** Ensure the **Tailwind** design is responsive and that error messages (Compilation Errors) are easy for students to read.

Role	Primary Focus	Key Tasks
Member 1: Systems & Infrastructure	The "Judge" & DevOps	Inngest setup, Docker sandboxing, Security, and Code Execution logic.

Member 2: Backend & Database	API & Role Logic	BetterAuth integration, MongoDB schemas, Admin/Setter workflows, and Contest logic.
Member 3: Frontend & UI	User Experience	Monaco Editor integration, Dashboards, Charts, and Video Player implementation.

QnA:

1. The "Why This Tech?" Questions

Q: Why the MERN stack? Why not PHP/MySQL or Django?

- **The Answer:** "MERN offers a **Unified Language Stack** (JavaScript/TypeScript), allowing for faster development and easier context switching between frontend and backend.
- **JSON Flexibility:** **MongoDB** is a NoSQL database, which is perfect for our 'Problem' model. Problem constraints and test cases vary in structure, and MongoDB allows us to store these as nested JSON without complex table joins.
- **Performance:** **Node.js** uses an event-driven, non-blocking I/O model, which is ideal for handling the multiple concurrent connections we expect during live contests."

Q: Why use Inngest? Can't you just run the code in a simple Express route?

- **The Answer:** "Running code is a **long-running task**. If we run it directly in a standard HTTP request, the connection might timeout, and it would block the main thread, making the site unresponsive for other users.
- **Reliability:** **Inngest** acts as our 'Reliability Layer.' It ensures that even if the server restarts or a worker fails, the submission isn't lost—it stays in the queue and retries automatically until it's processed."

2. The "Security & Logic" Questions (The Hard Ones)

Q: How will you prevent a user from submitting a virus like `system("rm -rf /")`?

- **The Answer:** "Security is our top priority. We are using **Docker Containers** for execution. The student's code never runs on our actual server. It runs in an isolated 'sandbox' with:
 - **No Network Access:** The code cannot call external APIs or hack other sites.
 - **Limited Resources:** We cap the RAM at 256MB and CPU at 0.5 cores to prevent 'Fork Bombs' or resource exhaustion.
 - **ReadOnly File System:** The code cannot modify or delete any system files."

Q: How will you handle an infinite loop in a student's code?

- **The Answer:** "We implement a strict **Time Limit Exceeded (TLE)** monitor. Our background worker (Inngest) sets a timeout (e.g., 2 seconds). If the Docker container doesn't finish within that window, the worker kills the process and updates the database with a 'TLE' verdict."

3. The "Feature & Role" Questions

Q: Why separate 'Problem Setters' and 'Admins'?

- **The Answer:** "This is for **Quality Control**. We want to encourage students to contribute problems, but we cannot allow unverified content to go public immediately. The **Admin** acts as a moderator to ensure the problem description is clear and the test cases are accurate before students can solve them."

Q: Why did you choose Cloudinary for videos instead of storing them in your own folder?

- **The Answer:** "Video files are heavy. Storing them on our own server would consume massive disk space and slow down our backups. **Cloudinary** provides a specialized **Content Delivery Network (CDN)** that optimizes video streaming based on the student's internet speed, ensuring a smooth experience without taxing our main backend."

4. The "Scaling" Question

Q: What happens if 500 students join a contest at the same time?

- **The Answer:** "Our architecture is **Decoupled**.
 - The API server just takes the code and puts it in the **Inngest Queue**. This takes milliseconds.
 - The 'Judging Workers' pull from that queue at their own pace.
 - The frontend uses **Polling** or **WebSockets** to check when the result is ready.
 - This ensures the website stays fast for everyone, even if the 'Judge' takes a few extra seconds to clear the backlog."